```
!pip install git+https://github.com/andreinechaev/nvcc4jupyter.git
%load_ext nvcc4jupyter
```

```
Collecting git+https://github.com/andreinechaev/nvcc4jupyter.git
  Cloning https://github.com/andreinechaev/nvcc4jupyter.git to /tmp/pip-req-build-iqeravrs
  Running command git clone --filter=blob:none --quiet https://github.com/andreinechaev/nvcc4jupyter.git /tmp/p
  Resolved https://github.com/andreinechaev/nvcc4jupyter.git to commit 173e636d9a3cb81c6101a160c1c8c86864eaaf5b
  Installing build dependencies ... done
  Getting requirements to build wheel ... done
  Preparing metadata (pyproject.toml) ... done
Building wheels for collected packages: nvcc4jupyter
  Building wheel for nvcc4jupyter (pyproject.toml) ... done
  Created wheel for nvcc4jupyter: filename=nvcc4jupyter-1.2.2-py3-none-any.whl size=12543 sha256=8f65034aebde6b
  Stored in directory: /tmp/pip-ephem-wheel-cache-j9yfrp45/wheels/2f/1e/79/584a8d85ad018ecc510baa90d4fbc161bd56
Successfully built nvcc4jupyter
Installing collected packages: nvcc4jupyter
Successfully installed nvcc4jupyter-1.2.2
Detected platform "Colab". Running its setup...
Source files will be saved in "/tmp/tmpca09s943".
```

```
%%cuda
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>
#include <sys/time.h>
#include <cuda_runtime.h>

#ifndef _COMMON_H
#define _COMMON_H

#define CHECK(call)                                                    \
{                                                                      \
    const cudaError_t error = call;                                    \
    if (error != cudaSuccess)                                          \
    {                                                                  \
        fprintf(stderr, "Error: %s:%d, ", __FILE__, __LINE__);         \
```

```
        fprintf(stderr, "code: %d, reason: %s\n", error,                    \
                cudaGetErrorString(error));                                 \
        exit(1);                                                            \
    }                                                                       \
}

inline double seconds()
{
    struct timeval tp;
    gettimeofday(&tp, NULL);
    return ((double)tp.tv_sec + (double)tp.tv_usec * 1.e-6);
}

#endif // _COMMON_H

// Check the results
void checkResult(float *hostRef, float *gpuRef, const int N)
{
    double epsilon = 1.0E-5;
    bool match = true;

    for (int i = 0; i < N; i++)
    {
        if (abs(hostRef[i] - gpuRef[i]) > epsilon)
        {
            match = false;
            printf("Arrays do not match!\n");
            printf("host %5.2f gpu %5.2f at index %d\n",
                   hostRef[i], gpuRef[i], i);
            break;
        }
    }

    if (match)
        printf("Arrays match.\n\n");
}

// Initialize host data
void initialData(float *ip, int size)
```

```
{
    srand((unsigned int)time(NULL));

    for (int i = 0; i < size; i++)
    {
        ip[i] = (float)(rand() & 0xFF) / 10.0f;
    }
}

// CPU vector addition
void sumArraysOnHost(float *A, float *B, float *C, const int N)
{
    for (int idx = 0; idx < N; idx++)
    {
        C[idx] = A[idx] + B[idx];
    }
}

// CUDA kernel for GPU vector addition
__global__ void sumArraysOnGPU(float *A, float *B, float *C, int N)
{
    unsigned int idx = blockIdx.x * blockDim.x + threadIdx.x;

    if (idx < N)
    {
        C[idx] = A[idx] + B[idx];
    }
}

int main(int argc, char **argv)
{
    printf("%s Starting...\n", argv[0]);

    // Set up the CUDA device
    int dev = 0;
    cudaDeviceProp deviceProp;

    CHECK(cudaGetDeviceProperties(&deviceProp, dev));
    printf("Using Device %d: %s\n", dev, deviceProp.name);
```

```
    CHECK(cudaSetDevice(dev));

    // Set vector size
    int nElem = 1 << 24;
    printf("Vector size %d\n", nElem);

    size_t nBytes = (size_t)nElem * sizeof(float);

    // Allocate host memory
    float *h_A = (float *)malloc(nBytes);
    float *h_B = (float *)malloc(nBytes);
    float *hostRef = (float *)malloc(nBytes);
    float *gpuRef = (float *)malloc(nBytes);

    if (h_A == NULL || h_B == NULL ||
        hostRef == NULL || gpuRef == NULL)
    {
        fprintf(stderr, "Host memory allocation failed!\n");
        return EXIT_FAILURE;
    }

    double iStart, iElaps;

    // Initialize host data
    iStart = seconds();
    initialData(h_A, nElem);
    initialData(h_B, nElem);
    iElaps = seconds() - iStart;

    printf("initialData Time elapsed %f sec\n", iElaps);

    memset(hostRef, 0, nBytes);
    memset(gpuRef, 0, nBytes);

    // Perform vector addition on CPU
    iStart = seconds();
    sumArraysOnHost(h_A, h_B, hostRef, nElem);
    iElaps = seconds() - iStart;
```

```
    printf("sumArraysOnHost Time elapsed %f sec\n", iElaps);

    // Allocate device memory
    float *d_A, *d_B, *d_C;

    CHECK(cudaMalloc((void **)&d_A, nBytes));
    CHECK(cudaMalloc((void **)&d_B, nBytes));
    CHECK(cudaMalloc((void **)&d_C, nBytes));

    // Copy input arrays from host to device
    CHECK(cudaMemcpy(d_A, h_A, nBytes, cudaMemcpyHostToDevice));
    CHECK(cudaMemcpy(d_B, h_B, nBytes, cudaMemcpyHostToDevice));

    // Configure CUDA kernel execution
    int iLen = 512;

    dim3 block(iLen);
    dim3 grid((nElem + block.x - 1) / block.x);

    // Execute the GPU kernel
    iStart = seconds();

    sumArraysOnGPU<<<grid, block>>>(d_A, d_B, d_C, nElem);

    // Check for kernel launch errors
    CHECK(cudaGetLastError());

    // Wait for the GPU kernel to finish
    CHECK(cudaDeviceSynchronize());

    iElaps = seconds() - iStart;

    printf("sumArraysOnGPU <<< %d, %d >>> Time elapsed %f sec\n",
           grid.x, block.x, iElaps);

    // Copy the GPU result back to host
    CHECK(cudaMemcpy(gpuRef, d_C, nBytes, cudaMemcpyDeviceToHost));

    // Compare CPU and GPU results
```

```
    checkResult(hostRef, gpuRef, nElem);

    // Free device memory
    CHECK(cudaFree(d_A));
    CHECK(cudaFree(d_B));
    CHECK(cudaFree(d_C));

    // Free host memory
    free(h_A);
    free(h_B);
    free(hostRef);
    free(gpuRef);

    return 0;
}
```

```
/tmp/tmpca09s943/8300411b-936e-4876-b986-405efb1c4322/cuda_exec.out Starting...
Using Device 0: Tesla T4
Vector size 16777216
initialData Time elapsed 0.719913 sec
sumArraysOnHost Time elapsed 0.054901 sec
sumArraysOnGPU <<< 32768, 512 >>> Time elapsed 0.000959 sec
Arrays match.
```

```
!nvcc --version
```

```
nvcc: NVIDIA (R) Cuda compiler driver
Copyright (c) 2005-2025 NVIDIA Corporation
Built on Wed_Aug_20_01:58:59_PM_PDT_2025
Cuda compilation tools, release 13.0, V13.0.88
Build cuda_13.0.r13.0/compiler.36424714_0
```

Start coding or generate with AI.